ЛАБОРАТОРНАЯ РАБОТА: КОНТЕЙНЕРИЗАЦИЯ ПРИЛОЖЕНИЯ TODOAPP (DOCKER)

Цель работы: Упаковать многослойное .NET приложение и базу данных PostgreSQL в изолированные Docker-контейнеры. Итогом работы должна стать инфраструктура, которая разворачивается на любом компьютере одной командой: `docker compose up --build`.

> **ОБРАТИТЕ ВНИМАНИЕ:**
>
> В вашем репозитории уже есть готовая папка `scripts/` с необходимыми bash-скриптами. Вам **не нужно** писать их с нуля! Ваша задача — правильно написать `Dockerfile` и `docker-compose.yml`, чтобы использовать эти скрипты. Выполняйте шаги строго последовательно и не пропускайте промежуточные проверки.

ШАГ 1: БЕЗОПАСНОСТЬ И ПЕРЕМЕННЫЕ ОКРУЖЕНИЯ

Хардкодить пароли в конфигурационных файлах Docker — грубое нарушение безопасности. Docker Compose умеет читать переменные из внешнего файла.

1. Создайте в корне проекта файл `.env`.
2. Запишите в него переменные для базы данных: `DB_USER=postgres`, `DB_PASSWORD=secret` (или придумайте свой), `DB_NAME=todo_db`.
3. Создайте копию этого файла и назовите `.env.example`. Оставьте в нем только ключи, без паролей (например, `DB_PASSWORD=`).
4. Убедитесь, что файл `.env` добавлен в ваш `.gitignore`, чтобы пароль случайно не попал в историю Git.

ШАГ 2: ИЗУЧЕНИЕ ГОТОВЫХ СКРИПТОВ

Прежде чем писать Dockerfile, загляните в папку `scripts/` в вашем проекте. Там лежат готовые инструменты для автоматизации:

- **`make-migration-bundle.sh`** — этот скрипт использует утилиту `dotnet ef` для генерации единого исполняемого файла миграций (`efbundle`). Мы вызовем его *во время сборки* образа.
- **`apply-migrations.sh`** — этот скрипт сначала запускает `efbundle` для обновления базы данных, а затем выполняет `exec dotnet TodoApp.Web.dll` для старта самого приложения. Мы укажем его как `ENTRYPOINT` *при запуске* контейнера.

ШАГ 3: DOCKERFILE (МНОГОЭТАПНАЯ СБОРКА)

Создайте файл `Dockerfile` в корне проекта. Мы используем Multi-stage build: собираем тяжелым SDK, а запускаем легким Runtime.

Этап 1: Builder (Сборка)

1. Начните с базового образа `mcr.microsoft.com/dotnet/sdk` (версии вашего проекта). Назовите этап `AS builder`. Установите рабочую директорию `WORKDIR /src`.
2. **Кэширование слоев:** Скопируйте `Directory.Packages.props` и все файлы `.csproj`, сохраняя структуру папок проекта. Выполните `dotnet restore` для главного проекта `TodoApp.Web.csproj`.
3. Скопируйте все остальные исходники (папку `src/`) и выполните сборку (`dotnet publish`) в папку `/app/publish` с флагом `--no-restore`.
4. **Подготовка EF Core:** Прямо в Dockerfile установите инструмент `dotnet-ef` глобально и добавьте путь к нему в системную переменную `PATH`.
   Подсказка: `ENV PATH="$PATH:/root/.dotnet/tools"`
5. Скопируйте скрипт `scripts/make-migration-bundle.sh` внутрь контейнера, выдайте ему права на исполнение (`chmod +x`) и запустите (команда `RUN`). Он создаст файл `efbundle`.

Этап 2: Runtime (Запуск)

1. Начните новый этап с базового образа `mcr.microsoft.com/dotnet/aspnet`. Установите

ШАГ 4: НЕВИДИМЫЙ ВРАГ (ОШИБКА CRLF)

Bash-скрипты в папке `scripts/` были скачаны из Git. Если Git на вашем компьютере с Windows настроен по умолчанию, он скачал эти файлы с окончаниями строк `CRLF` (`\r\n`). А Linux внутри Docker понимает только `LF` (`\n`) и "спотыкается" о символ возврата каретки (`\r`).

Как вылечить репозиторий раз и навсегда:

1. Создайте файл `.editorconfig` в корне проекта. Пропишите в нем правила для вашей IDE:

   ```
   root = true
   [*]
   end_of_line = lf
   insert_final_newline = true
   ```

2. Создайте файл `.gitattributes`. Это скажет самому Git всегда использовать LF для текстовых файлов:

   ```
   * text eol=lf
   *.bat text eol=crlf
   *.cmd text eol=crlf
   ```

3. Выполните в терминале команды, которые заставят Git найти старые сломанные файлы в репозитории и автоматически заменить в них CRLF на LF:

   ```
   git add --renormalize .
   git commit -m "Normalize line endings to LF"
   ```

4. Запустите `docker build -t todoapp-test . --no-cache` еще раз. Теперь сборка скриптов должна пройти без ошибок!

ШАГ 5: ОРКЕСТРАЦИЯ (DOCKER COMPOSE)

Создайте файл `docker-compose.yml`. Здесь мы опишем, как база данных и наш бэкенд должны работать вместе.

5.1 Сервис базы данных (db)

1. Определите сервис `db`. Образ: `postgres:15-alpine`.
2. В блоке `environment` передайте переменные Postgres (`POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`), подставив в них значения из вашего файла `.env` (синтаксис: `${DB_USER}`).
3. Добавьте блок `volumes`. Примонтируйте внутреннюю папку Postgres (`/var/lib/postgresql/data`) к именованному тому (например, `postgres_data`), чтобы база не стиралась при остановке контейнера.
4. Добавьте блок `healthcheck`. Команда для проверки: `pg_isready -U ${DB_USER} -d ${DB_NAME}`. Укажите интервал и количество попыток.

5.2 Сервис бэкенда (api)

1. Определите сервис `api`. В блоке `build` укажите путь к папке с вашим Dockerfile (`.`).
2. Пробросьте порты (например, `8080:8080`).
3. Добавьте `depends_on` на сервис `db`. Обязательно укажите `condition: service_healthy`, чтобы бэкенд не пытался запуститься до того, как база данных будет полностью готова принимать подключения.
4. **САМОЕ ВАЖНОЕ (СТРОКА ПОДКЛЮЧЕНИЯ):** В локальном `appsettings.json` у вас написано `Host=localhost`. В Docker это не сработает! Для контейнера бэкенда `localhost` — это он сам.
   В блоке `environment` сервиса `api` переопределите строку подключения: `ConnectionStrings__DefaultConnection: "Host=db;Database=${DB_NAME};Username=${DB_USER};Password=${DB_PASSWORD}"`.
   (Обратите внимание, что мы используем имя сервиса `db` в качестве Host).

> **Что гуглить, если запутались в синтаксисе:**
> `docker compose postgres volume example`, `docker compose depends_on condition service_healthy`.

ШАГ 6: ФИНАЛЬНЫЙ ЗАПУСК И ПРОВЕРКА

Откройте терминал в корне проекта.

1. Выполните команду: `docker compose up --build`.
2. Внимательно следите за логами в консоли. Вы должны увидеть следующий порядок событий:
   - Сначала скачивается и запускается Postgres.
   - Бэкенд ждет (состояние waiting).
   - Как только Postgres сообщает "database system is ready to accept connections", стартует процесс бэкенда.
   - Скрипт `apply-migrations.sh` запускает `efbundle`, который применяет миграции к базе.
   - Запускается само веб-приложение (надпись "Application started").
3. Откройте браузер и перейдите по адресу `http://localhost:8080/swagger` (или по тому порту, который вы пробросили). Если открылась документация API и вы можете создать Todo-задачу — поздравляем, вы успешно контейнеризировали приложение!